JAVA PROGRAMMING

Comprehensive Basic to Intermediate Student Notes

Subject: Core Java Fundamentals

Target Audience: Beginners, Computer Science Students, & Software Engineering Candidates

Language Version: Java 17 LTS / Standard Edition

Document Type: Technical Reference Guide & Quick Study Notes

Table of Contents

1. Introduction to Java Architecture & JVM internal structure
2. Basic Syntax, Variables & Data Types
3. Operators & Control Flow Statements
4. Object-Oriented Programming (OOP) Fundamentals
5. Pillars of OOP: Encapsulation, Inheritance, Polymorphism, Abstraction
6. Arrays & Strings Deep Dive
7. Exception Handling Mechanism
8. Java Collections Framework (JCF)
9. File I/O & Basic Streams
10. Java 8+ Modern Features (Lambdas & Streams)

1. Introduction to Java Architecture

Java is a high-level, class-based, object-oriented programming language designed to have as few implementation dependencies as possible. It follows the WORA (Write Once, Run Anywhere) principle, enabling Java code to execute on any device that supports the Java Virtual Machine (JVM).

1.1 JDK, JRE, and JVM Breakdown

Understanding the distinction between JVM, JRE, and JDK is foundational to Java programming:

- **JVM (Java Virtual Machine):** An abstract machine that provides a runtime environment in which Java bytecode can be executed. It converts bytecode into machine-specific instructions.
- **JRE (Java Runtime Environment):** The software package that provides Java class libraries along with the JVM and other components to run applications written in Java.
- **JDK (Java Development Kit):** A full-featured software development kit that includes the JRE, compilers (like `javac`), archivers (`jar`), documentation generators (`javadoc`), and other tools needed for development.

> **Key Takeaway:** JDK = JRE + Development Tools | JRE = JVM + Library Classes

1.2 JVM Architecture & Memory Management

When a Java program runs, the JVM manages memory allocation divided into specific regions:

- **Classloader:** Loads class files into memory.
- **Method Area:** Stores class structures, static variables, and method bytecode.
- **Heap Area:** Allocates memory for runtime objects and instance variables. Managed by the Garbage Collector.
- **Stack Area:** Stores local variables, reference variables, and partial call stacks per thread.
- **PC Registers:** Holds the address of the currently executing JVM instruction per thread.
- **Native Method Stack:** Holds instructions for native C/C++ libraries.

1.3 Hello World Compilation Flow

```java
// HelloWorld.java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Execution Flow:

1. Write source code in `HelloWorld.java`.
2. Compile using `javac HelloWorld.java` to generate `HelloWorld.class` (Bytecode).
3. Run using `java HelloWorld` which loads bytecode into the JVM.

2. Basic Syntax, Variables & Data Types

2.1 Primitive Data Types

Java is a strongly-typed language. Primitive types store raw values directly in memory (typically on the stack).

Type	Size	Default Value	Range / Value
`byte`	1 byte (8 bits)	0	-128 to 127
`short`	2 bytes (16 bits)	0	-32,768 to 32,767
`int`	4 bytes (32 bits)	0	-2^{31} to $2^{31}-1$
`long`	8 bytes (64 bits)	0L	-2^{63} to $2^{63}-1$
`float`	4 bytes (32 bits)	0.0f	Single-precision floating point
`double`	8 bytes (64 bits)	0.0d	Double-precision floating point
`char`	2 bytes (16 bits)	''	0 to 65,535 (Unicode)
`boolean`	1 bit (JVM dependent)	false	true or false

2.2 Variable Types in Java

- **Local Variables:** Declared inside methods, constructors, or blocks. No default value; must be initialized before use.
- **Instance Variables:** Declared inside a class but outside methods. Initialized when an instance is created.
- **Static (Class) Variables:** Declared with the `static` keyword inside a class. Shared across all instances.

```
public class VariableDemo {
    // Static variable
    public static int staticCounter = 0;

    // Instance variable
    private int instanceID;

    public void process() {
        // Local variable
        int localVar = 10;
        System.out.println("Local variable: " + localVar);
    }
}
```

2.3 Type Casting

Converting one data type into another is known as type casting.

Implicit Casting (Widening Conversion)

Occurs automatically when converting a smaller type to a larger type.

```
int num = 100;
double dNum = num; // Automatic casting from int to double
```

Explicit Casting (Narrowing Conversion)

Requires explicit manual casting operator when converting a larger type to a smaller type. Risk of data loss!

```
double d = 105.75;
int i = (int) d; // Value becomes 105 (decimal truncated)
```

3. Operators & Control Flow Statements

3.1 Java Operators

Java provides a rich set of operators to manipulate variables:

- **Arithmetic:** `+`, `-`, `*`, `/`, `%`
- **Relational:** `==`, `!=`, `>`, `<`, `>=`, `<=`
- **Logical:** `&&` (AND), `||` (OR), `!` (NOT)
- **Bitwise:** `&`, `|`, `^`, `~`, `<<`, `>>`
- **Assignment:** `=`, `+=`, `-=`, `*=`, `/=`
- **Ternary:** `condition ? value_if_true : value_if_false`

3.2 Conditional Statements

```java
int score = 85;

// If-Else If-Else Structure
if (score >= 90) {
    System.out.println("Grade A");
} else if (score >= 80) {
    System.out.println("Grade B");
} else {
    System.out.println("Grade C");
}

// Switch Statement
char grade = 'B';
switch (grade) {
    case 'A':
        System.out.println("Excellent");
        break;
    case 'B':
        System.out.println("Good");
        break;
    default:
        System.out.println("Keep trying");
}
```

3.3 Looping Statements

```java
// 1. Standard For Loop
for (int i = 0; i < 5; i++) {
    System.out.println("Iteration: " + i);
}

// 2. Enhanced For Loop (For-Each)
int[] numbers = {10, 20, 30, 40};
for (int n : numbers) {
    System.out.println("Value: " + n);
}

// 3. While Loop
int count = 0;
while (count < 3) {
    System.out.println("Count: " + count);
    count++;
}

// 4. Do-While Loop (Executes at least once)
int x = 10;
do {
    System.out.println("Executed at least once");
    x++;
} while (x < 5);
```

4. Object-Oriented Programming (OOP) Fundamentals

Java is built around Object-Oriented Programming (OOP) principles, structured around objects that contain data fields and methods.

4.1 Class vs Object

- **Class:** A user-defined blueprint or template from which objects are created. It defines state (fields) and behavior (methods).
- **Object:** A basic unit of OOP representing a real-world entity with state, behavior, and identity. Created at runtime in Heap memory.

4.2 Constructors

Constructors are special methods invoked when an object is instantiated. They must have the same name as the class and no return type.

```
public class Car {
    private String brand;
    private int year;

    // Default Constructor
    public Car() {
        this.brand = "Unknown";
        this.year = 2020;
    }

    // Parameterized Constructor
    public Car(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    public void displayInfo() {
        System.out.println("Brand: " + brand + ", Year: " + year);
    }
}
```

4.3 Memory Allocation of Objects

> **Memory Note:** Reference variables are stored on the *Stack* pointing to actual object instances allocated on the *Heap*.

```
// Creation of Object
Car myCar = new Car("Toyota", 2022);
```

5. Pillars of Object-Oriented Programming

5.1 Encapsulation

Encapsulation is the practice of bundling fields and methods into a single unit and restricting access to implementation details via access modifiers.

```
public class BankAccount {
    // Private fields hide state
    private double balance;

    // Getter
    public double getBalance() {
        return balance;
    }

    // Setter with validation
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
}
```

5.2 Inheritance

Inheritance allows a child class (subclass) to inherit attributes and methods from a parent class (superclass), promoting code reusability. Uses keyword `extends`.

```
// Parent Class
class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}

// Child Class
class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}
```

5.3 Polymorphism

Polymorphism allows objects to be treated as instances of their parent class while executing overridden subclass implementations.

1. Compile-time Polymorphism (Method Overloading)

```
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
}
```

2. Runtime Polymorphism (Method Overriding)

```
class Shape {
    void draw() { System.out.println("Drawing Shape"); }
}

class Circle extends Shape {
    @Override
    void draw() { System.out.println("Drawing Circle"); }
}
```

5.4 Abstraction

Hiding complex implementation details and showing only key functionality. Achieved using **Abstract Classes** or **Interfaces**.

```
// Interface definition
interface PaymentProcessor {
    void processPayment(double amount);
}

class CreditCardPayment implements PaymentProcessor {
    public void processPayment(double amount) {
        System.out.println("Processed Credit Card payment: $" + amount);
    }
}
```

6. Arrays & Strings Deep Dive

6.1 Arrays in Java

An array is a fixed-size homogeneous collection of elements stored in contiguous memory locations.

```
// Array Declaration & Initialization
int[] numbers = new int[5];
numbers[0] = 10;
numbers[1] = 20;

// Multidimensional Arrays
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
```

6.2 Java String Concept & String Pool

In Java, `String` objects are **immutable** (cannot be changed once created). They are stored in a special heap area called the *String Constant Pool (SCP)* to optimize memory usage.

```
String s1 = "Hello"; // Created in String Constant Pool
String s2 = "Hello"; // Points to same reference in SCP
String s3 = new String("Hello"); // Explicit Heap allocation

System.out.println(s1 == s2); // true (same reference)
System.out.println(s1 == s3); // false (different references)
System.out.println(s1.equals(s3)); // true (content comparison)
```

6.3 StringBuilder vs StringBuffer

When frequent string modification is required, use mutable classes:

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread Safety	Thread-Safe	Not Thread-Safe	Thread-Safe (Synchronized)
Performance	Slow on modification	Fastest	Slower than StringBuilder

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" Notes");
sb.reverse();
System.out.println(sb.toString());
```

7. Exception Handling Mechanism

Exception Handling prevents unexpected runtime crashes and allows graceful degradation of programs.

7.1 Exception Hierarchy

All exception types are subclasses of the root class `java.lang.Throwable`.

- **Throwable**
  - **Error:** Irrecoverable failures (e.g., `OutOfMemoryError`, `StackOverflowError`).
  - **Exception:** Conditions that applications should handle.
    - **Checked Exceptions:** Checked at compile-time (e.g., `IOException`, `SQLException`).
    - **Unchecked Exceptions (RuntimeException):** Checked at runtime (e.g., `NullPointerException`, `ArithmeticException`).

7.2 Try-Catch-Finally Block Structure

```
try {
    int result = 10 / 0; // Throws ArithmeticException
} catch (ArithmeticException e) {
    System.err.println("Error: Division by zero! " + e.getMessage());
} catch (Exception e) {
    System.err.println("Generic exception catch");
} finally {
    // Always runs regardless of exception occurrence
    System.out.println("Cleanup logic executing here.");
}
```

7.3 Throw vs Throws

Keyword	Purpose	Syntax Position
`throw`	Used to explicitly throw a single exception object.	Inside method body
`throws`	Declares exceptions that a method might throw up the call stack.	In method signature

```
public void validateAge(int age) throws IllegalArgumentException {
    if (age < 18) {
        throw new IllegalArgumentException("Underage access forbidden!");
    }
}
```

8. Java Collections Framework (JCF)

The Java Collections Framework provides a set of interfaces and classes to store and manipulate groups of objects efficienty.

8.1 JCF Hierarchy Overview

- **Collection (Interface)**
  - **List (Ordered, allows duplicates):** `ArrayList`, `LinkedList`, `Vector`
  - **Set (Unordered, unique elements):** `HashSet`, `LinkedHashSet`, `TreeSet`
  - **Queue (FIFO order):** `PriorityQueue`, `ArrayDeque`
- **Map (Key-Value pairs, unique keys):** `HashMap`, `LinkedHashMap`, `TreeMap`

8.2 Core Collections Code Example

```
import java.util.*;

public class CollectionDemo {
    public static void main(String[] args) {
        // 1. List Example
        List<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Apple"); // Duplicates allowed

        // 2. Set Example
        Set<String> set = new HashSet<>(list); // Removess duplicates

        // 3. Map Example
        Map<String, Integer> map = new HashMap<>();
        map.put("Alice", 25);
        map.put("Bob", 30);

        for (Map.Entry<String, Integer> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " => " + entry.getValue());
        }
    }
}
```

8.3 Performance Comparison

Data Structure	Access Time	Insertion Time	Deletion Time
ArrayList	O(1)	O(N)	O(N)
LinkedList	O(N)	O(1)	O(1)
HashSet	O(1)	O(1)	O(1)
TreeMap	O(log N)	O(log N)	O(log N)

9. File I/O & Basic Streams

Java I/O API enables reading and writing data to external sources such as files, networks, or arrays.

9.1 Byte Streams vs Character Streams

- **Byte Streams (8-bit):** Classes like `FileInputStream` and `FileOutputStream` used for handling raw binary content (images, audio).
- **Character Streams (16-bit):** Classes like `FileReader`, `FileWriter`, `BufferedReader` used for text.

9.2 File Reading & Writing Example

```
import java.io.*;

public class FileIODemo {
    public static void main(String[] args) {
        // Writing to a file using Try-With-Resources
        try (BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt"))) {
            writer.write("Java File I/O Demonstration");
            writer.newLine();
            writer.write("Writing high performance buffered data.");
        } catch (IOException e) {
            e.printStackTrace();
        }

        // Reading from a file
        try (BufferedReader reader = new BufferedReader(new FileReader("output.txt"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

10. Java 8+ Modern Features

Java 8 introduced functional programming capabilities that drastically changed Java syntax.

10.1 Lambda Expressions

Lambda expressions provide concise implementation of Single Abstract Method (SAM) interfaces (Functional Interfaces).

```
// Traditional anonymous inner class
Runnable r1 = new Runnable() {
    public void run() {
        System.out.println("Thread running...");
    }
};

// Modern Java 8+ Lambda alternative
Runnable r2 = () -> System.out.println("Thread running via Lambda!");
```

10.2 Stream API

The Stream API allows processing sequences of elements in a declarative style using transformations and aggregations.

```
import java.util.*;
import java.util.stream.*;

public class StreamDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("John", "Alice", "Bob", "Andrew", "Charlie");

        // Filter names starting with 'A', convert to uppercase and sort
        List<String> result = names.stream()
            .filter(name -> name.startsWith("A"))
            .map(String::toUpperCase)
            .sorted()
            .collect(Collectors.toList());

        System.out.println(result); // Output: [ALICE, ANDREW]
    }
}
```

10.3 Core Cheatsheet Summary

Best Practices Summary:

- Prefer Interfaces over Concrete Classes for flexible designs.
- Always leverage Try-With-Resources for automatic closing of streams/readers.
- Use `StringBuilder` inside loops instead of String concatenation.
- Favor immutability wherever possible to make code safe in multithreaded environments.